

KDT-17 3rd Team Project

쓰레기 이미지 12 종 분류

데이터 품질에서 시작한 딥러닝 이미지 분류

3 조 (김이박)

팀장 이현중

팀원 김태경 · 박병찬

0 발표 구성

15 분 · 3 인 분담

1

김태경

데이터 이해

- 원본 데이터 현황과 클래스 불균형
- 이미지 품질 점검
- 정확도만 보면 안 되는 이유

5 분

2

이현중

정제와 모델링

- 중복 이미지 3 단계 탐지
- 라벨 오류 진단과 정정
- 모델 비교와 하이퍼파라미터

5 분

3

박병찬

평가와 해석

- 최종 성능과 클래스별 분석
- 오분류 패턴이 말하는 것
- 가설검정 · 운영 임계값

5 분

1

문제 정의

재활용 쓰레기 사진을 12 개 클래스로 자동 분류한다

데이터셋

- Kaggle mostafaabla/garbage-classification
- 원본 15,515 장 · 12 개 클래스
- battery, biological, brown-glass, cardboard,
- clothes, green-glass, metal, paper, plastic,
- shoes, trash, white-glass

왜 어려운가

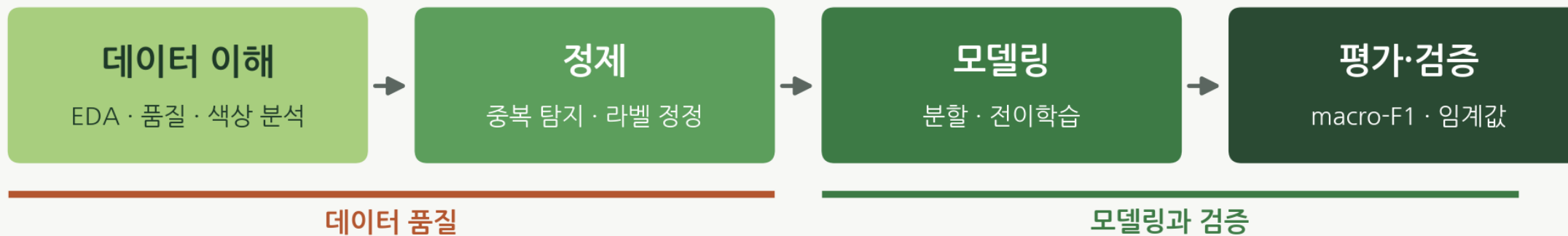
- 클래스 불균형이 심하다 — 최다 clothes 5,325 장 , 최소 brown-glass 607 장
- 유리 3 종은 형태가 같고 색만 다르다
- plastic / metal / paper 는 재질 경계가 사람에게도 모호하다
- 원본 데이터에 중복과 라벨 오류가 섞여 있다

네 가지 어려움 중 마지막 항목이 이 발표의 축이다 . 폴더 구조만 봐서는 알 수 없는 문제를 먼저 찾아냈고 , 그 과정을 파트 2 에서 다룬다 .

2

전체 파이프라인

네 단계 중 앞의 절반이 데이터 품질에 대한 것이다



데이터 품질 단계

원본을 그대로 학습에 넣지 않았다. 중복과 라벨 오류를 먼저 찾아내고, 그 근거를 문서로 남겼다.

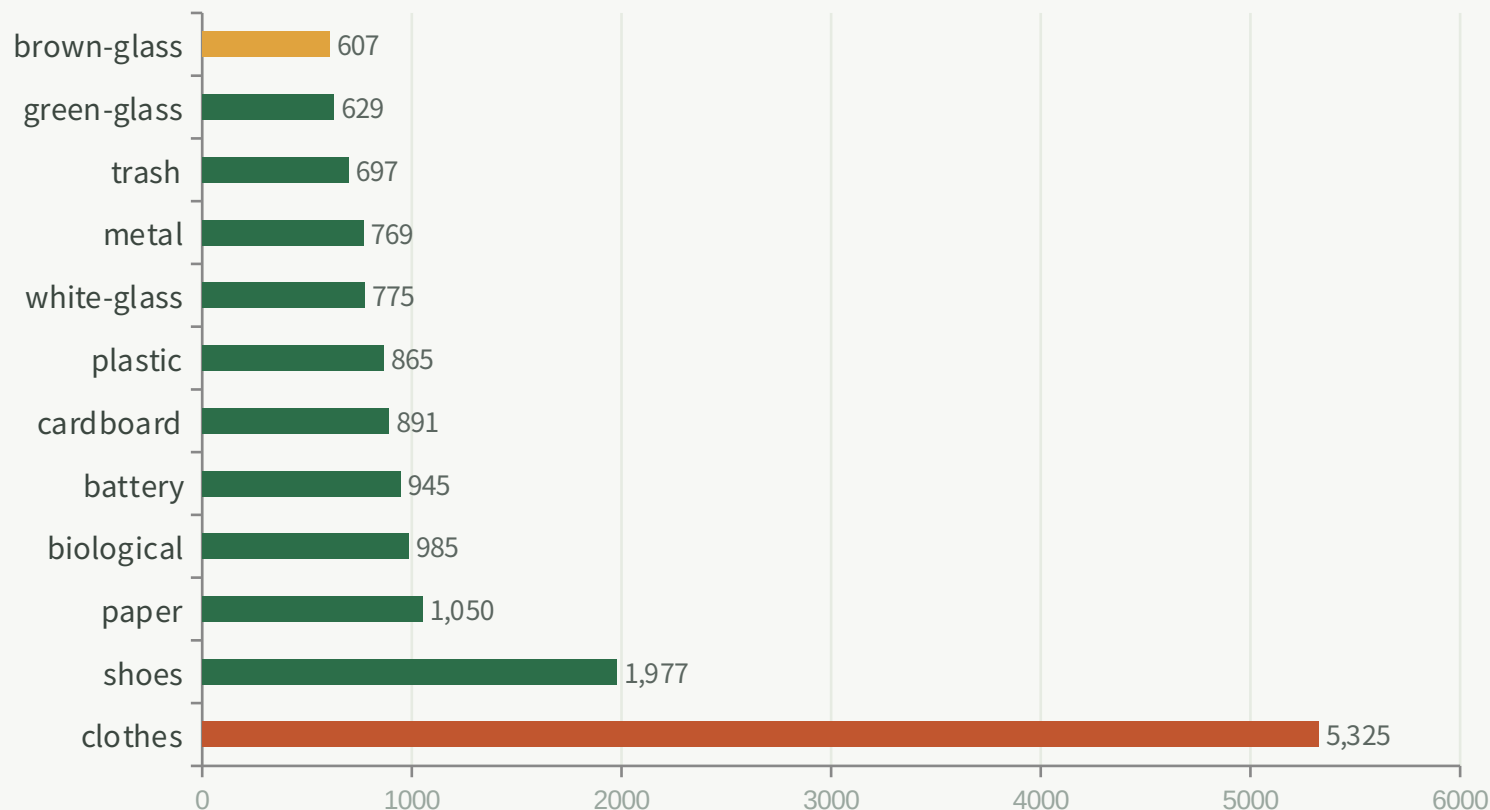
모델링 · 검증 단계

정제된 데이터 위에서 모델을 비교하고, 성능 숫자가 믿을 만한지 통계로 확인했다.

3

데이터 현황

클래스별 이미지 수 — 최다와 최소가 8.77 배 차이



34.3%

clothes 한 클래스가
전체에서 차지하는 비중

8.77 배

최다 / 최소 클래스 비율

clothes 한 클래스가 전체의 3분의 1이다. 이 불균형이 평가지표 선택을 결정한다 — 다음 슬라이드에서 그 이유를 직접 측정한다.

4

정확도만 보면 안 되는 이유

아무것도 학습하지 않은 모델을 실제로 측정했다

무지성 기준선

모든 입력을 clothes 라고만 답하는 모델

학습 없음 · 파라미터 0 개

정확도 0.3398

balanced accuracy 0.0833

macro-F1 0.0423

정확도는 34%
macro-F1 은 0.042

약 8 배 차이

정확도는 다수 클래스 하나만 잘 맞혀도 올라간다 .
macro-F1 은 12 개 클래스의 F1 을 동등 가중으로 평균하므로 ,
나머지 11 개를 전부 틀리면 즉시 무너진다 .

쓰레기 분류의 목적은 ' 많이 나오는 종류를 잘 맞히는 것 ' 이
아니라 ' 모든 종류를 고르게 맞히는 것 ' 이다 .

주 평가지표로 macro-F1 을 채택하고 accuracy · weighted-F1 · macro-AUC 는 참고로 병기한다 . 유비가 아니라 우리 데이터에서 직접 측정한 반례다 .

5

이미지 품질 점검

학습을 시작하기 전에 확인해야 할 것들

손상 파일

0 건

PIL verify 로 전수 검사했다.
학습 중 크래시 위험은 없다

팔레트 (P) 모드

34 장

채널이 3 이 아니라 배치가 깨진다.
`convert("RGB")` 가 필수다

해상도 산포

51 ~ 888px

종횡비도 제각각이다.
`resize` 가 필요하다 (128 / 224)

색상 정보

유리 3 종

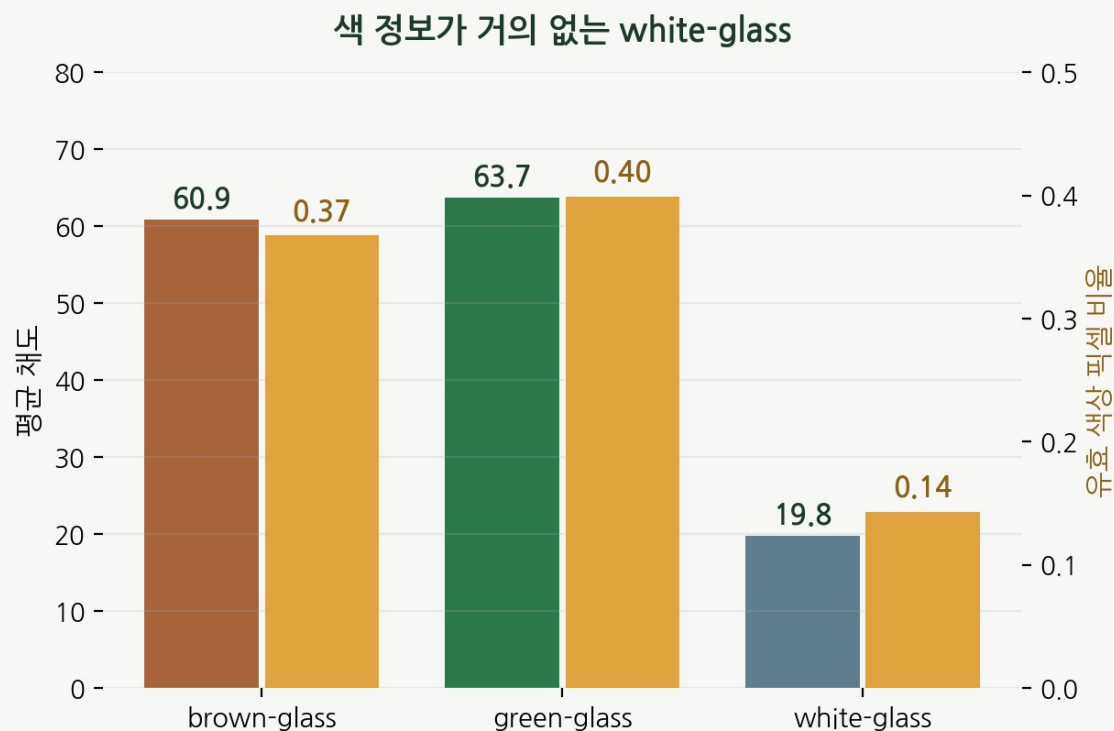
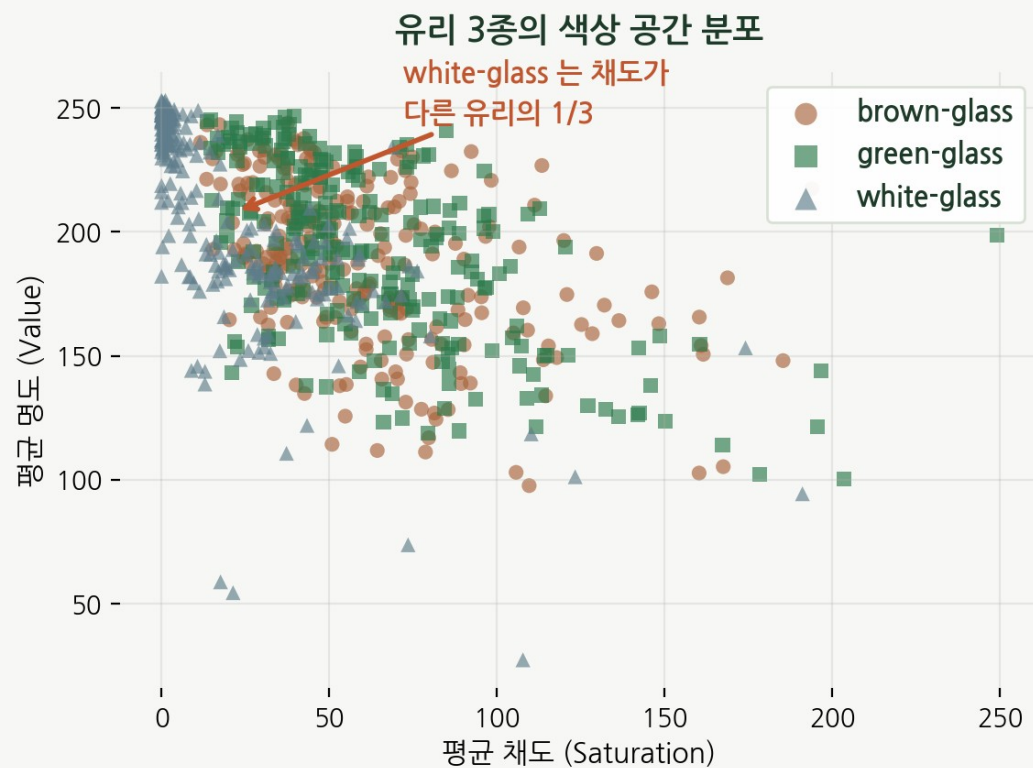
형태가 같아 색이 유일한 단서다.
증강에서 색을 건드리면 안 된다

네 항목 모두 감이 아니라 세어본 값이다 . P 모드 34 장이 실제로 있었기 때문에 `convert("RGB")` 는 관행이 아니라 근거 있는 처리다 .

6

유리 3 종의 색상 분석

형태가 같은 세 클래스를 가르는 단서는 색뿐이다



클래스당 250 장 표본을 HSV 로 변환하고 채도 40 미만, 명도 30 미만 픽셀을 뺐다 — 배경과 그림자 노이즈를 걸러내기 위해서다.
색이 정답 신호이므로 증강에서 hue = 0.0, saturation = 0.05 로 색을 거의 건드리지 않기로 결정했다.

7

검정 ① 색 차이는 우연이 아닌가

겹침 계수는 기술통계일 뿐, 우연의 가능성을 말해주지 않는다

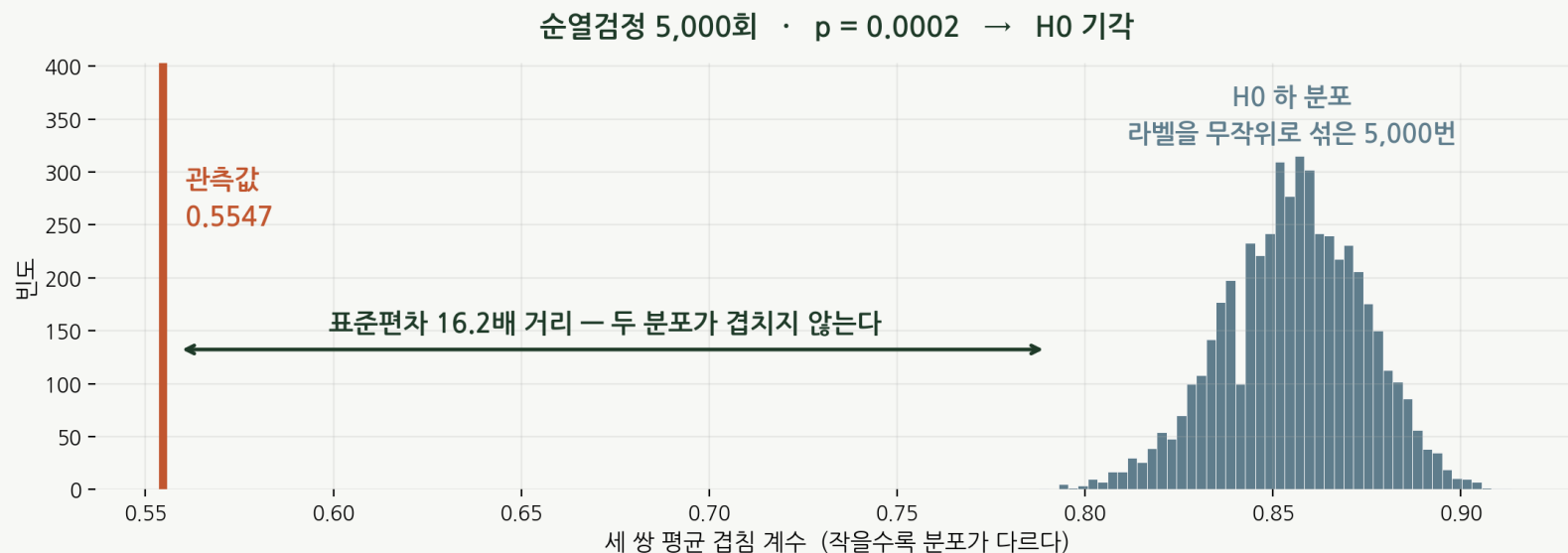
가설

H0 세 클래스의 색상 분포는 같다

H1 세 클래스의 색상 분포는 다르다

방법 — 순열검정

라벨을 무작위로 섞어 "차이가 없는 세상"을 5,000 번 만들고, 그 세상에서 관측된 만큼의 분리가 나올 확률을 센다. 분포 가정이 필요 없어 정규분포와 거리가 먼 색상값에 적합하다.



결론

$p = 0.0002$

H0 기각

색은 유리 3 종을 가르는 유효한 신호다.

증강에서 색을 흔들지 않기로 한 결정에 통계적 근거가 붙는다.

PART 2 · 이현중

폴더만 봐서는 알 수 없는 문제

같은 사진이 여러 장 들어 있고, 신발 사진이 clothes 폴더에 있었다

9

중복 이미지 탐지

1.2 억 쌍을 세 단계로 좁혀 들어간다

전체 이미지 쌍

약 1.2억 쌍

픽셀 단위 전수 비교는 불가능

dHash 후보

Hamming ≤ 5

구조가 비슷한 쌍만 후보로

픽셀 검증 통과

평균차 < 0.05

32×32 RGB 실제 값으로 재확인

최종 근접중복

42그룹

육안으로 오탐 7그룹 제외

MD5 만으로는 리사이즈·재저장된 같은 사진을 잡을 수 없다

18 그룹

완전 동일 (MD5)

3 그룹

라벨이 충돌

42 그룹

근접 중복

7 그룹

육안 오탐 제외

탐지 결과를 눈으로 확인한다

알고리즘이 찾은 것과 사람이 판단한 것

MD5로는 잡히지 않는 근접 중복

같은 그룹으로 묶인 세 장 · 파일 바이트는 서로 다르다

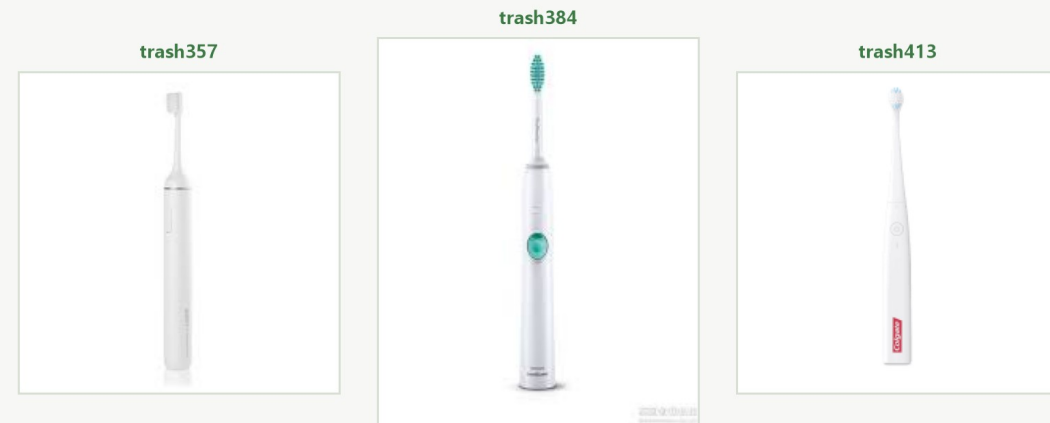


진짜 근접 중복

사람 눈에는 완전히 같은 사진인데 파일이 세 개다. 리사이즈나 재저장으로 바이트가 달라져 MD5로는 잡히지 않았다. 삭제하지 않고 같은 split에 묶었다.

알고리즘이 같다고 판정한 오탐

흰 배경 + 세로 물체 구도가 같을 뿐, 서로 다른 칫솔 제품이다



알고리즘의 오탐

dHash와 픽셀 검증을 모두 통과했지만 열어보니 서로 다른 칫솔 제품이었다. 흰 배경에 세로로 선 물체라는 제품 사진 구도가 같았을 뿐이다. 7그룹 30장을 제외했다.

라벨 오류 진단

모델을 데이터 검사 도구로 거꾸로 이용한다

착안점

모델이 " 확신 있게 틀린 " 예측은 둘 중 하나다

정말 헛갈리는 사진이거나, 원본 라벨이 잘못됐거나

왜 두 단계로 나뉘냐

train 데이터는 모델이 그 라벨 그대로 암기했기 때문에, 라벨이 틀려도 틀린 라벨을 그대로 재현해 " 틀림 " 으로 잡히지 않는다. 그래서 train 은 2-fold out-of-fold 로 " 안 본 상태 " 의 예측을 새로 만들어냈다.

진단 결과

1 단계 held-out

4,655 장 오분류 123 건

확신도 0.9 이상 59 건

2 단계 train OOF

10,838 장 오분류 382 건

확신도 0.9 이상 145 건

합계 204 건 전수 육안 확인

자동 추출은 후보 생성까지만. 라벨을 옮길지 뺄지는 전부 사람이 이미지를 직접 보고 판단했다.

13

실제로 어떤 사진이었나

모두 clothes 폴더에 들어 있던 파일이다

폴더가 알려주지 않는 라벨 오류

clothes 폴더에 들어 있던 신발 사진들 · 마지막 한 장은 반대 방향(반바지)

clothes3062 → shoes



clothes3330 → shoes



clothes5002 → shoes



clothes3545 → shoes



clothes2197 → shoes



shoes1826 → clothes



46 장

정정한 라벨 수

clothes → shoes 43

shoes → clothes 1

색상 · 재질 2

왜 중요한가

모델이 clothes 를 shoes 로 틀렸을 때 , 그건 모델의 실수가 아니라 정답이 잘못돼 있던 것이다 .

모델을 개선해도 줄지 않던 이유다 .

14 정제 결과

파일을 옮기거나 지우지 않고 manifest 한 장으로 관리했다

15,515

원본



- 31

제외



15,484

최종

46 장

라벨 정정

완전 동일 중복 여분

16 장

그룹당 1 장만 남긴다

라벨이 충돌한 중복

6 장

같은 사진에 다른 라벨이라 양쪽 모두 뺀다

애매한 경계 사례

3 장

사람이 봐도 판단이 갈린다

사진이 아닌 이미지

6 장

클립아트 · 목업 템플릿 · 스케치

라벨 정정

46 장

clothes → shoes 43, 그 외 3

근접 중복 그룹

25 그룹

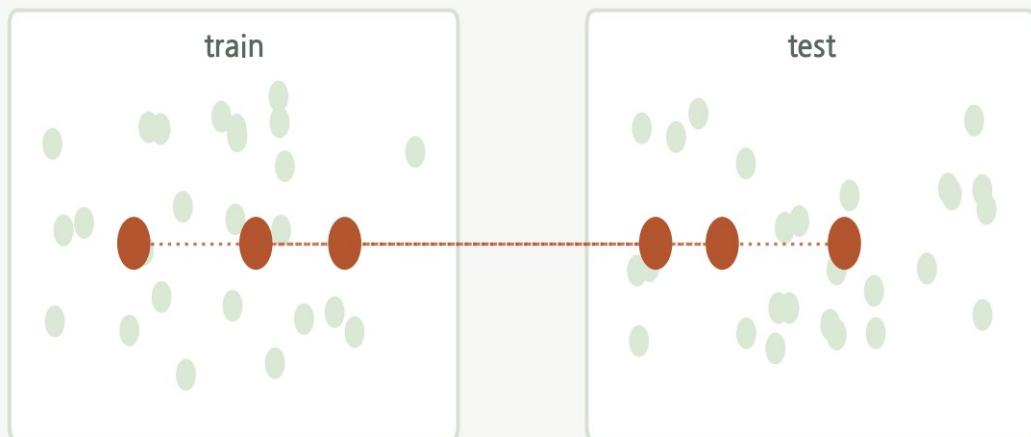
삭제하지 않고 같은 split 에 넣는다

manifest.csv 한 장으로 관리한다 — filepath, orig_label, final_label, group_id, status, reason. 파일을 옮기지 않으므로 언제든지 되돌릴 수 있고 근거가 남는다.

데이터 분할

같은 사진이 train 과 test 로 갈라지면 시험 문제를 미리 본 셈이다

단순 계층 분할 — 같은 사진이 갈라진다



13그룹 26장이 양쪽에 걸침 → test 점수 부풀려짐

그룹 인지 분할 — 통째로 한쪽에



같은 그룹은 항상 같은 쪽 → 갈라진 그룹 0개

MD5 는 근접 중복을 못 잡는다

바이트가 1 비트만 달라도 다른 파일로 본다 . 리사이즈하거나 다시 저장하면 해시가 완전히 달라진다 .

최종 분할 70 / 15 / 15

10,832

train

2,318

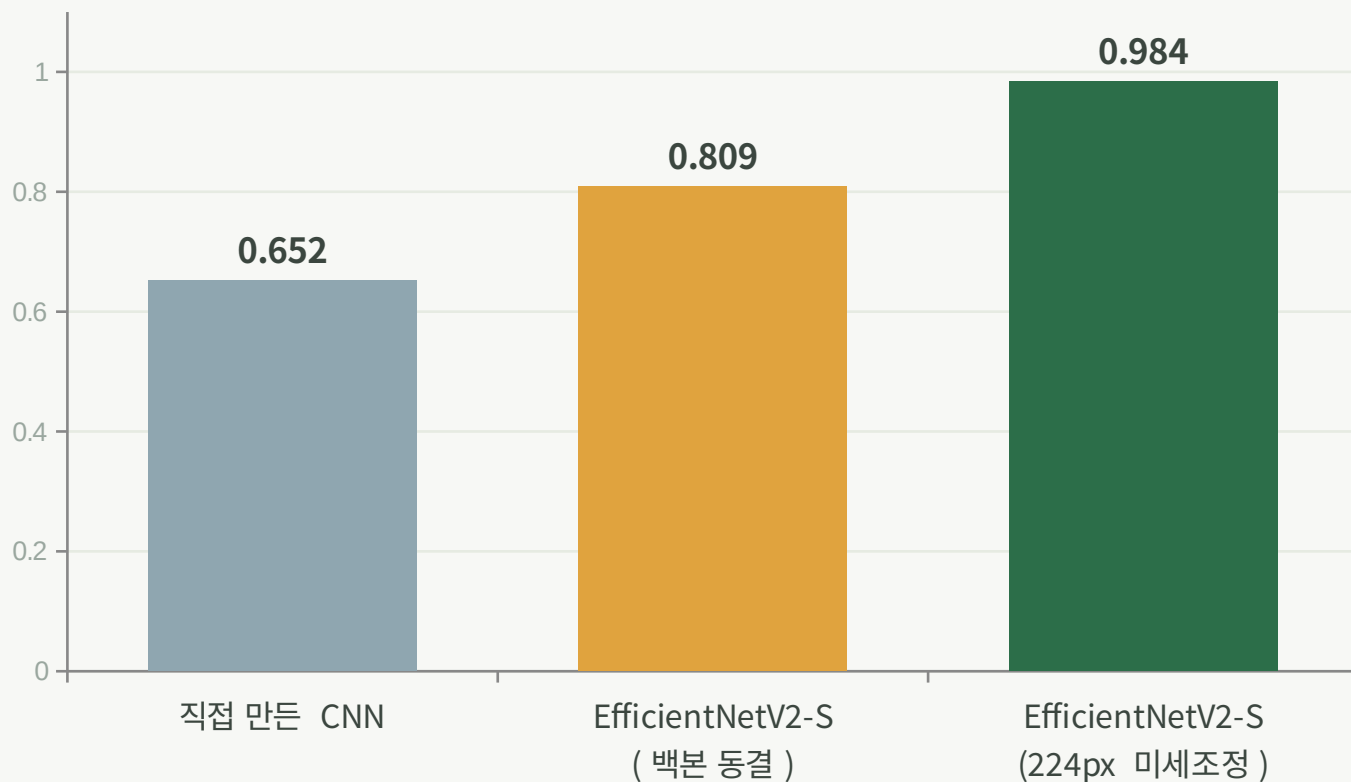
val

2,334

test

모델 비교

밑바닥 학습 vs 전이학습 — 검증셋 macro-F1 기준 · 입력 224px



전이학습의 효과

+0.33

ImageNet 128 만 장에서 배운 특징이
쓰레기 사진에도 그대로 통한다

학습률 비교

1e-4	0.9840
5e-5	0.9787
3e-4	0.9738

후보 3 개를 전부 끝까지 학습해 비교했다

최종 채택은 EfficientNetV2-S unfreeze · 입력 224px · lr 1e-4 다. 밑바닥 학습 대비 0.33 차이는 사전학습 특징이 그대로 통한다는 뜻이다.

시행착오에서 배운 것

128px 에서 최적이던 학습률이 224px 에서도 최적은 아니었다

처음 방법

128px 기준으로 고른 학습률

5e-5	0.9731 ← 128px 최적
1e-4	0.9600
3e-4	0.9658

그런데 입력을 224px 로 올린 뒤 같은 학습률을 그대로 쓰자 val macro-F1 이 0.9787 에 그쳤다 . 128px 에서 최적이던 값이 224px 에서도 최적은 아니었다 .

고친 방법

224px 에서 학습률을 다시 골랐다

1e-4	0.9840	채택
5e-5	0.9787	128px 최적값
3e-4	0.9738	

조건이 바뀌면 하이퍼파라미터도 다시 골라야 한다 . 해상도를 올렸으면 학습률도 다시 탐색해야 한다는 뜻이다 .

PART 3 · 박병찬

숫자가 진짜인지 확인하는 단계

test 는 여기서 단 한 번만 사용한다



19

최종 성능

EfficientNetV2-S (unfreeze, 224px, lr 1e-4) · test 2,334 장

0.9859

accuracy

0.9787

macro-F1

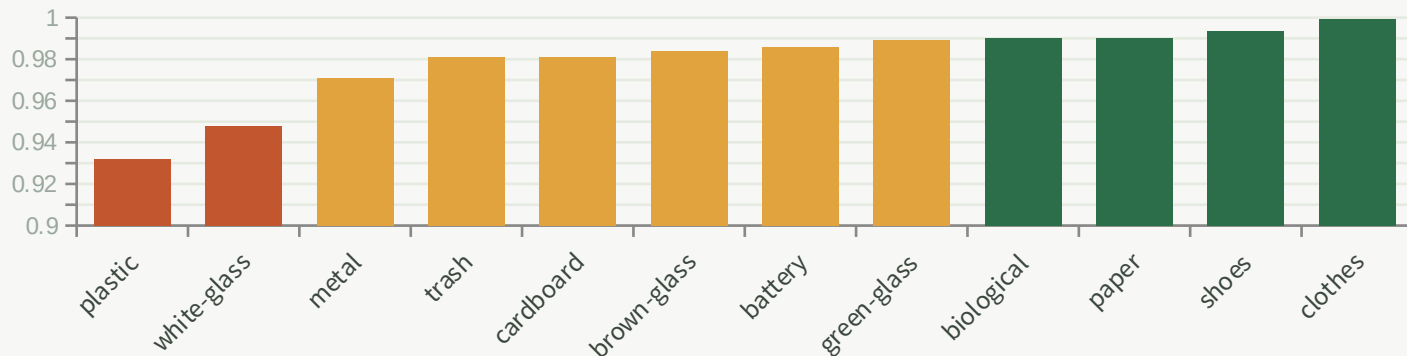
0.9790

macro-recall

0.9998

macro-AUC

클래스별 F1



가장 약한 세 클래스

plastic 0.932 (정밀도 0.918)
white-glass 0.948 (재현율 0.940)
metal 0.971 (정밀도 0.943)

재질 경계가 모호한 클래스들이다.
정제로 해결되는 문제가 아니다.

모델 선정은 검증셋 macro-F1 으로만 했고 test 는 이 한 번만 썼다. 남은 약점은 재질 경계가 모호한 클래스에 몰려 있다.

20

검정 ② 두 지표의 격차는 유의한가

accuracy 0.9859 과 macro-F1 0.9787 의 차이가 표본 변동일 수는 없는가

가설

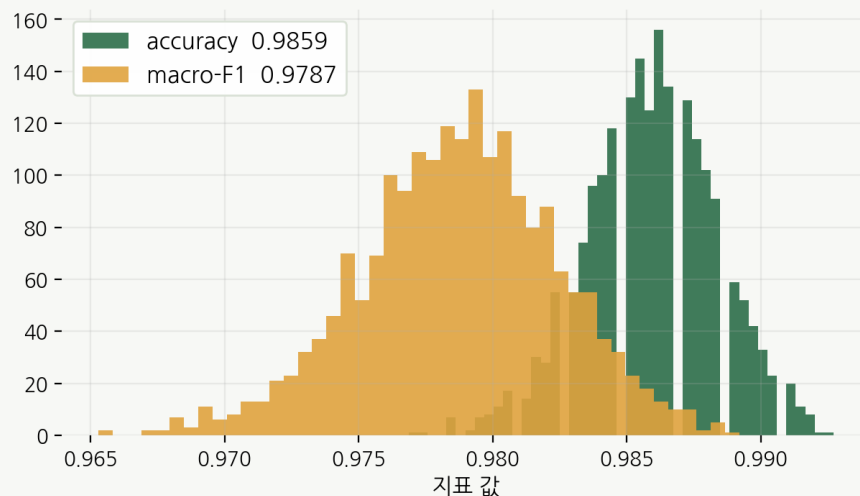
H0 두 지표의 차이는 0 이다

H1 accuracy 가 더 크다

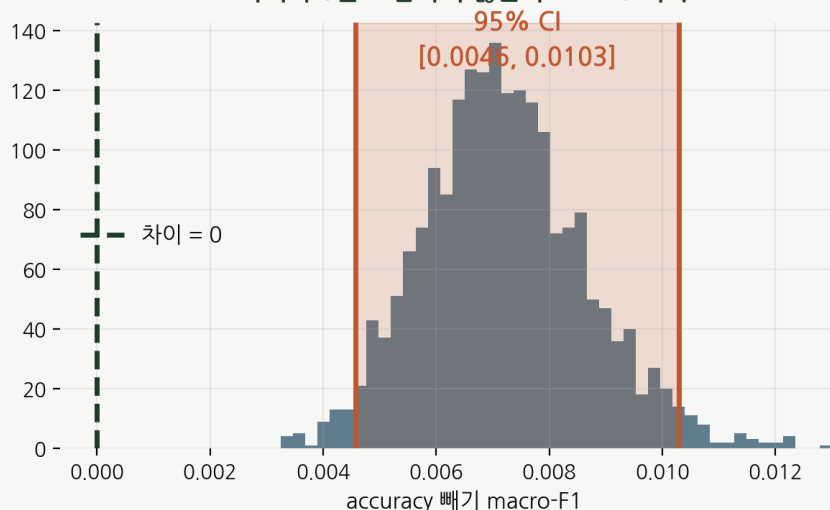
방법 — 부트스트랩

test 2,334 장에서 복원추출로 같은 크기의 표본을 2,000 번 만들고 , 각 표본에서 두 지표의 차이를 계산한다 . 그 분포의 95% 신뢰구간이 0 을 포함하지 않으면 H0 를 기각한다 .

두 지표의 부트스트랩 분포



차이가 0을 포함하지 않는다 → H0 기각



결론

하한 0.0046

H0 기각

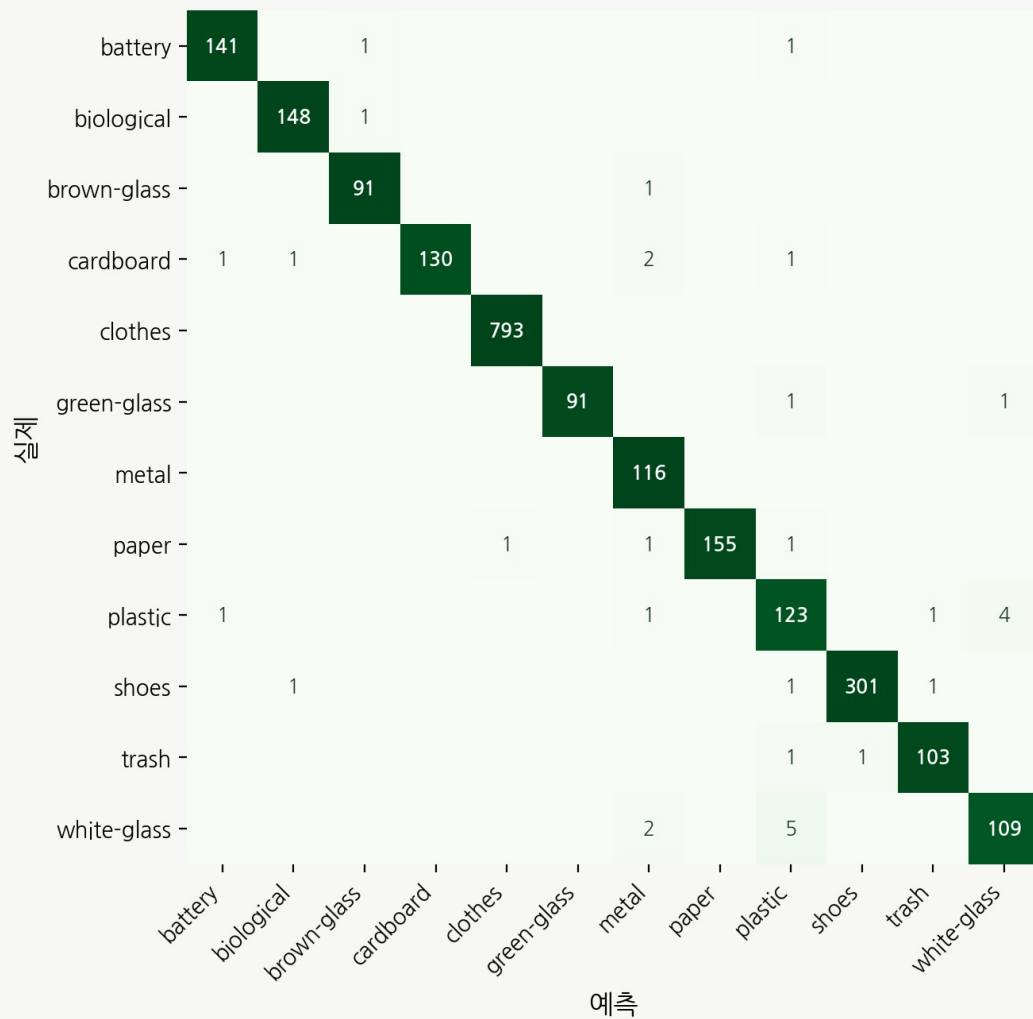
2,000 번 재표본 중 두 지표가 뒤집힌 경우가 없었다 .

정확도가 macro-F1 보다 큰 것은 우연이 아니라 구조적 차이다 .

무엇을 무엇으로 틀렸나

2,334 장 중 33 건 · 대각선이 정답, 나머지가 오분류

test 혼동행렬 · 2,334장 중 오분류 33건



읽는 법

- 행 = 실제 라벨
- 열 = 모델 예측
- 진할수록 그 행에서 차지하는 비중이 크다
- 대각선 밖 숫자가 오분류

눈에 띄는 것

대각선이 압도적으로 진하다.
clothes 793 장을 전부 맞혔다.

남은 33 건은 재질 · 색상 혼동에 몰려 있다.
white-glass ↔ plastic 이 9 건으로 최다이고,
나머지는 metal 관련이 대부분이다.

오분류 패턴이 말하는 것

정제 효과는 정확도 숫자가 아니라 틀리는 방식에서 드러났다

정제 전

clothes → shoes 가 실험 내내 1 위

20 건대를 계속 유지했다. 모델을 아무리 개선해도 줄지 않았는데, 원인이 모델이 아니라 원본 라벨이었기 때문이다.

정제 후 상위 오분류

white-glass → plastic	5 건
plastic → white-glass	4 건
cardboard → metal	2 건
white-glass → metal	2 건
clothes → shoes	0 건

EDA 단계에서 이미 확인한 위험이었다

평균 채도 / 유효 색상 픽셀 비율

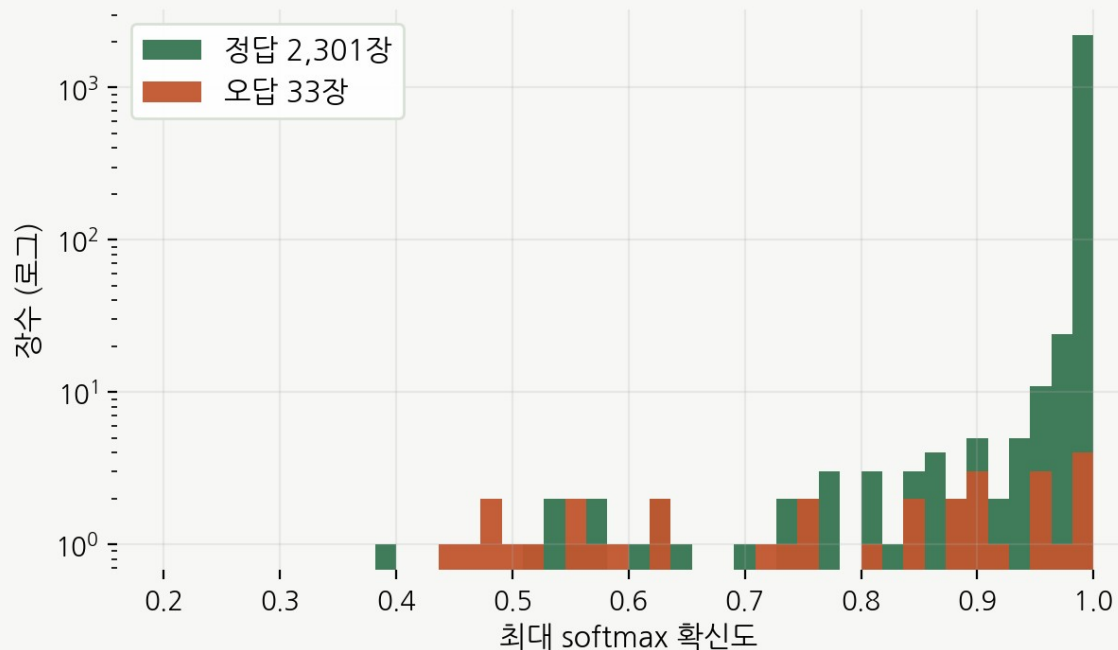
brown-glass	60.85	0.37
green-glass	63.68	0.40
white-glass	19.78	0.14

EDA 에서 white-glass 의 채도가 다른 유리의 3 분의 1 이라는 것을 확인했다. 색 정보가 없으면 질감과 반사에 의존하게 되고 금속 광택과 겹칠 수 있다 — 이 위험을 미리 파악한 상태에서 모델링에 들어갔고, 실제 오분류가 그 예상과 일치했다.

모델은 얼마나 확신하는가

이 분포가 운영 임계값을 정하는 근거가 된다

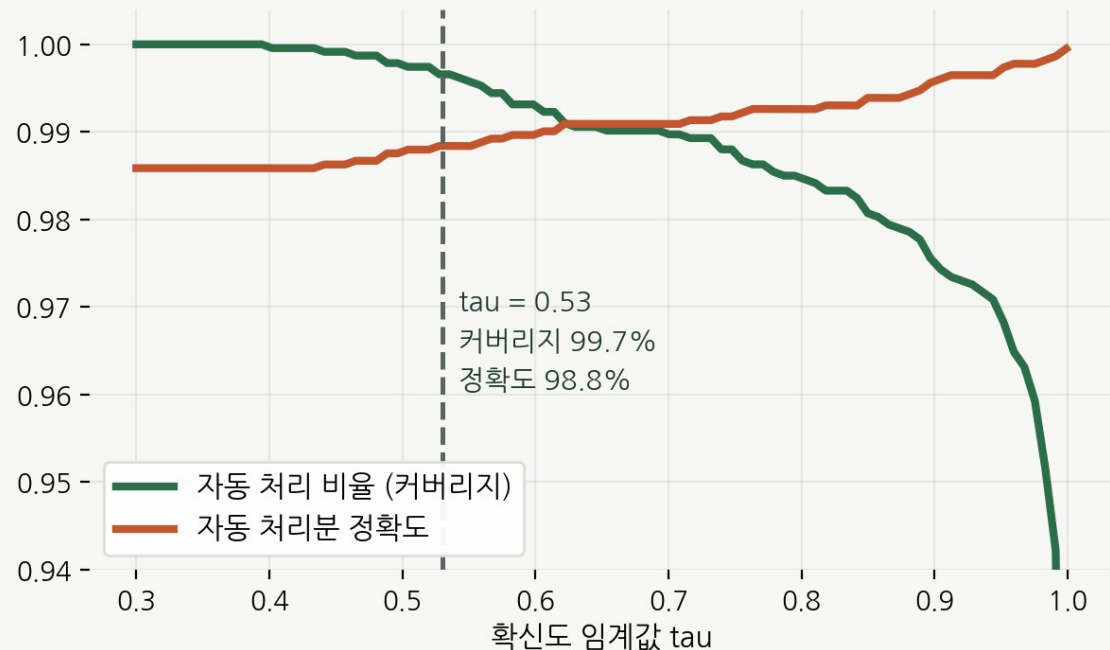
모델은 대부분 확신하고, 확신할 때는 맞는다



정답일 때와 오답일 때가 갈린다

정답 2,301 장은 확신도 1.0 근처에 몰려 있고, 오답 33 건은 넓게 퍼져 있다.

얼마를 사람이 확인할 것인가



운영으로 옮기면

0.3%(8 장) 만 사람이 확인하면 나머지는 98.8% 정확도로 자동 분류된다.

검정 ③④ 임계값 조정은 효과가 있었나

선택은 검증셋에서, 적용은 test 에 한 번만

검정 ③ 클래스별 가중 argmax

소수 클래스에 유리한 가중치를 곱해 argmax 를 밀었다. 검증셋에서 고른 최적 계수는 0.00 이었다.

	조정 맞춤	조정 틀림
기본 맞춤	2,301	0
기본 틀림	0	33

불일치 0 쌍 — 예측이 한 장도 바뀌지 않아 검정이 성립하지 않는다

검정 ④ 확신도 기반 판단 보류

확신도가 tau 미만이면 사람에게 넘긴다. 자동 처리 구간의 정확도가 전체보다 유의하게 높은지 부트스트랩으로 확인했다.

전체 정확도	0.9859
자동 처리 정확도	0.9884
차이 95% CI	[0.0008, 0.0047]
0 을 포함하지 않는다 — H0 기각, 보류 규칙이 작동한다	

여기서 가장 중요한 것은 ③ 이 실패했다는 사실

검증셋에서는 가중치가 macro-F1 을 올렸다. 그런데 test 에서는 예측이 한 장도 바뀌지 않았다. 이득이 검증셋에 과적합되어 있었다는 뜻이다.

만약 test 확률에서 임계값을 골랐다면 반드시 미세한 개선이 나왔을 것이고, 그 개선은 허구였을 것이다. 검증셋에서 고르고 test 에 한 번만 적용했기 때문에 이 사실을 확인할 수 있었다.

결론

모델을 바꾸는 것보다
데이터를 고치는 게 먼저였다

0.652 → 0.984

모델을 바꿔서 얻은 것

53 건 → 33 건

해상도를 올려서 얻은 것

clothes → shoes 오분류는 데이터를 고쳐서 20 건대에서 0 건이 됐다

아직 못 한 것

1 탐지 임계값은 임의값이다

Hamming 5, 픽셀차 0.05 는 시행착오로 정했다. 값을 바꾸려면 매번 재학습이 필요하다.

2 라벨 검수가 고르지 않다

모델이 확신하며 틀린 것만 후보로 올리므로, 못 배우는 유형의 오류는 놓친다.

3 단일 시드 단일 학습이다

0.005 안팎의 차이는 실행 간 변동 폭 안에 있을 수 있다.

주요 주장 네 개에 통계 검정을 붙였고, 그중 하나는 효과가 없다는 결과가 나왔다. 그것도 결과로 보고한다 — 감추면 나머지 세 개도 믿을 수 없기 때문이다.